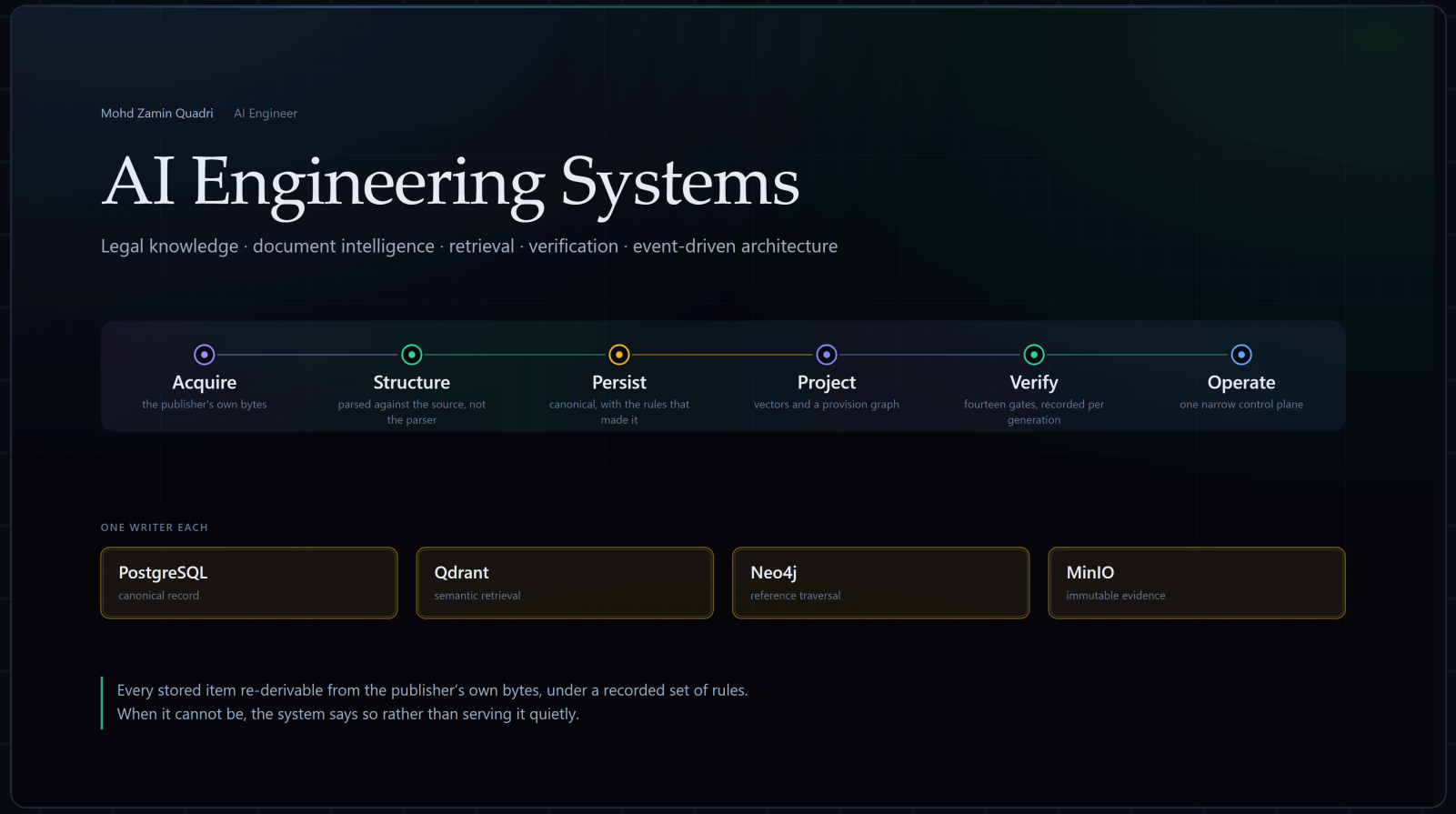
AI Engineering Systems



- Mohd Zamin Quadri — AI Engineer, BP-ITCS
- Legal knowledge infrastructure · document intelligence · retrieval · verification
- Four of five services implemented; three more extended

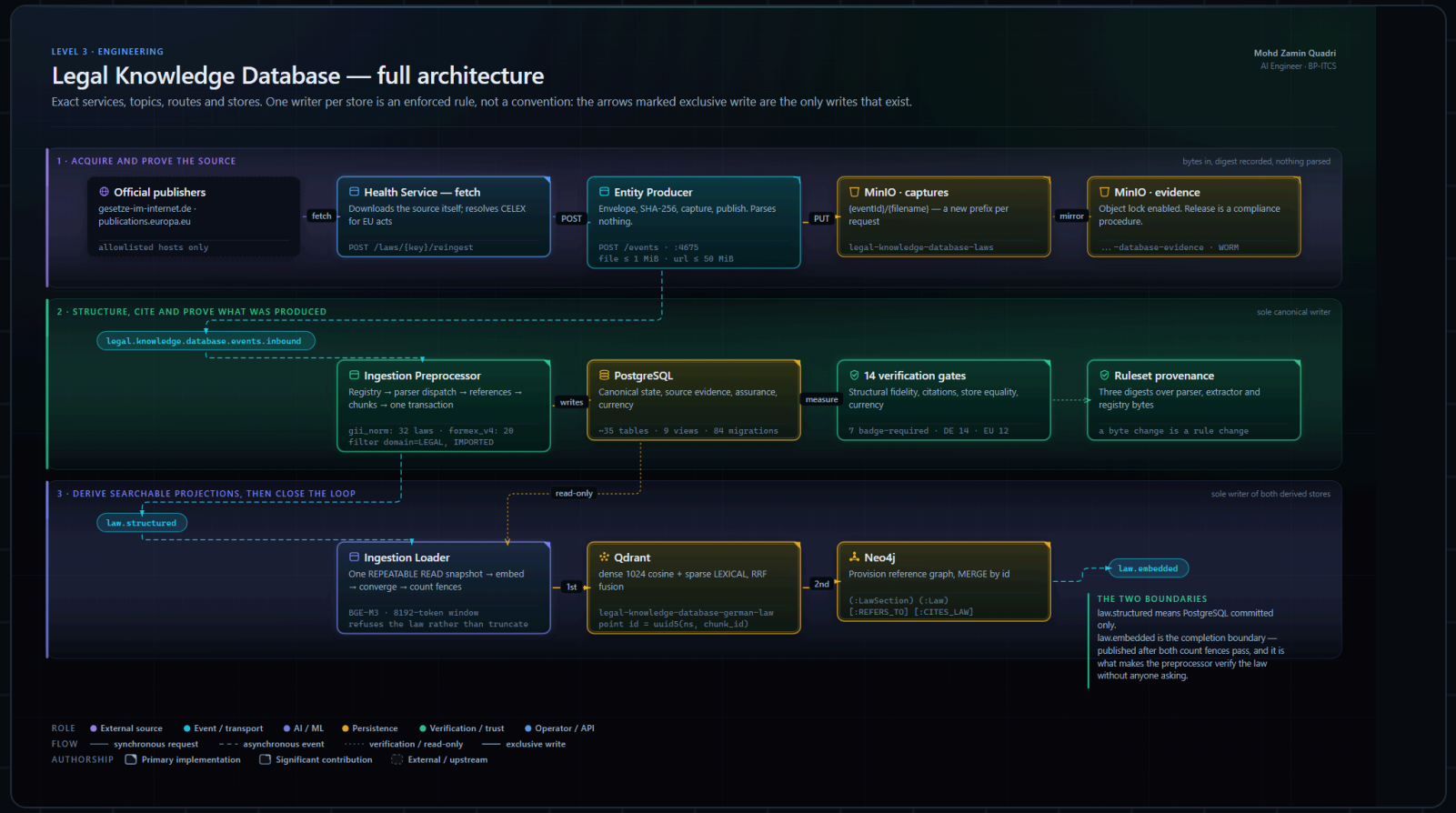
What was actually hard about it?

From legal source to trusted machine knowledge

- Authoritative law changes without telling you, in two unrelated XML markup families
- Extract it for retrieval and you hold three representations that can each drift from the source
- So the problem was provenance and operational control — not retrieval

How does it fit together?

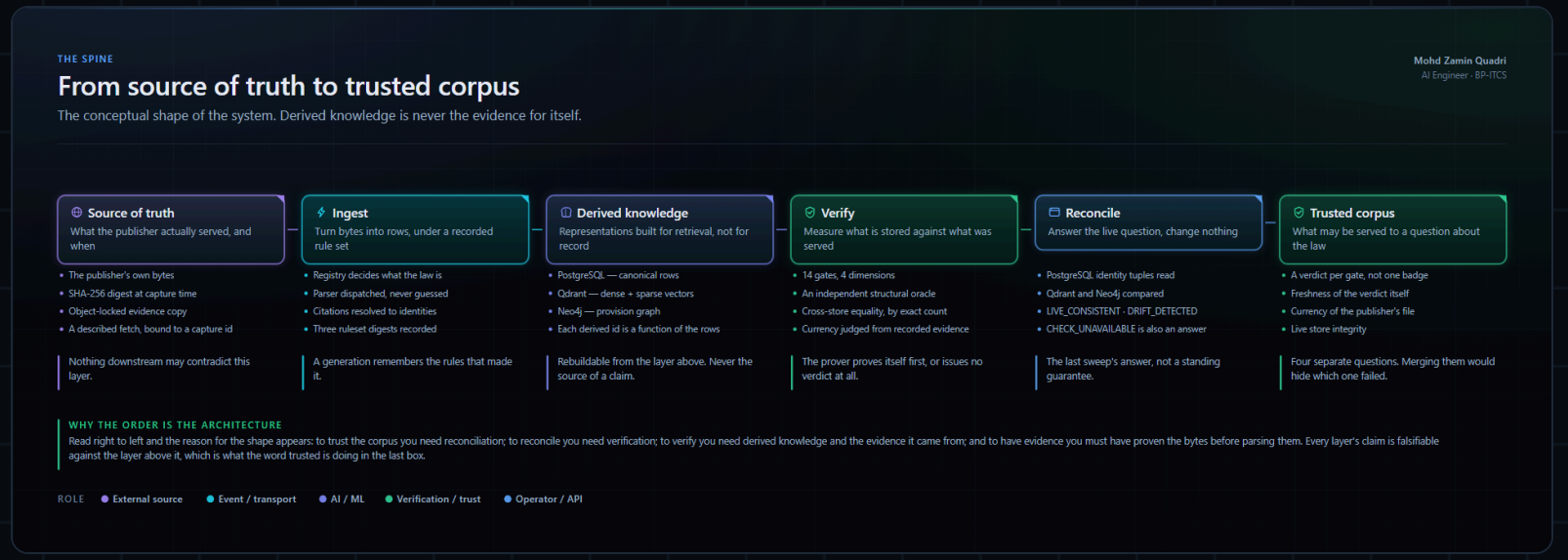
End-to-end legal knowledge architecture



- Five services: a three-stage ingestion pipeline, plus a two-part control plane
- Three Kafka topics; one law maps to one partition, so generations cannot race
- Exactly one service writes each store — enforced by deleting the path that violated it

What happens to one law?

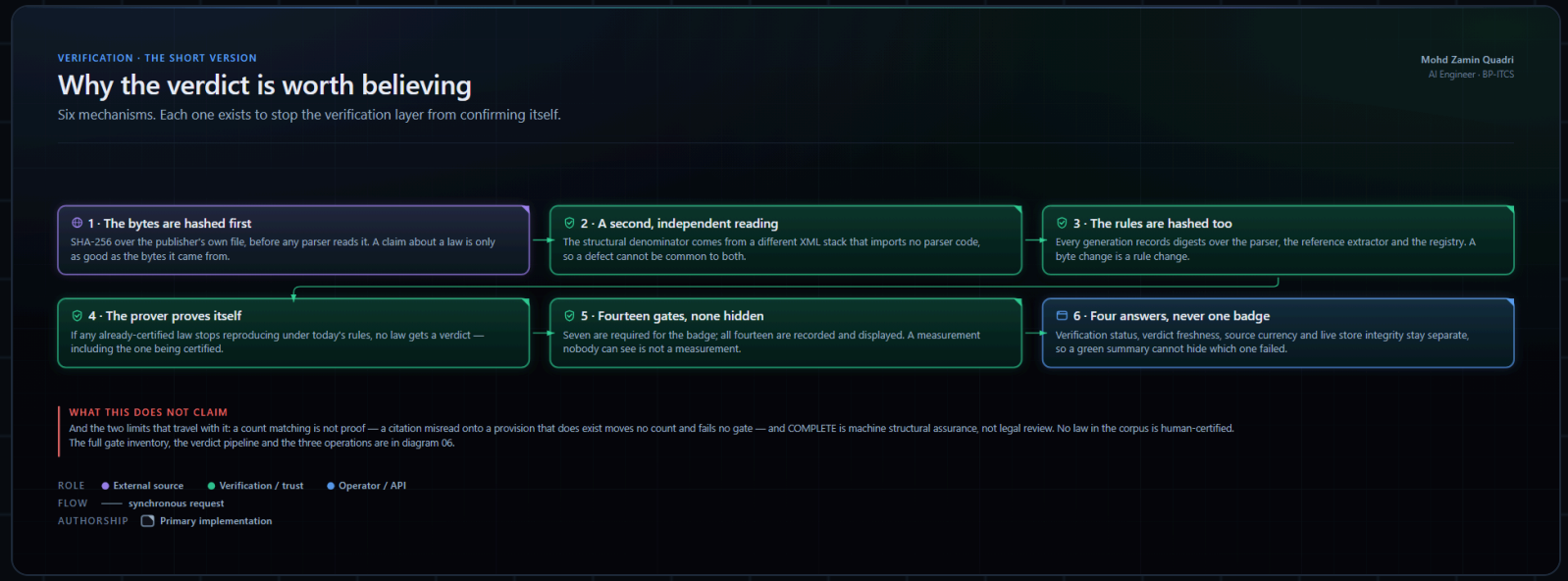
Source → trusted corpus



- Bytes are hashed before anything interprets them
- The structural denominator is built independently of the parser
- law.structured means PostgreSQL only; law.embedded is the completion boundary

How do you know it is right?

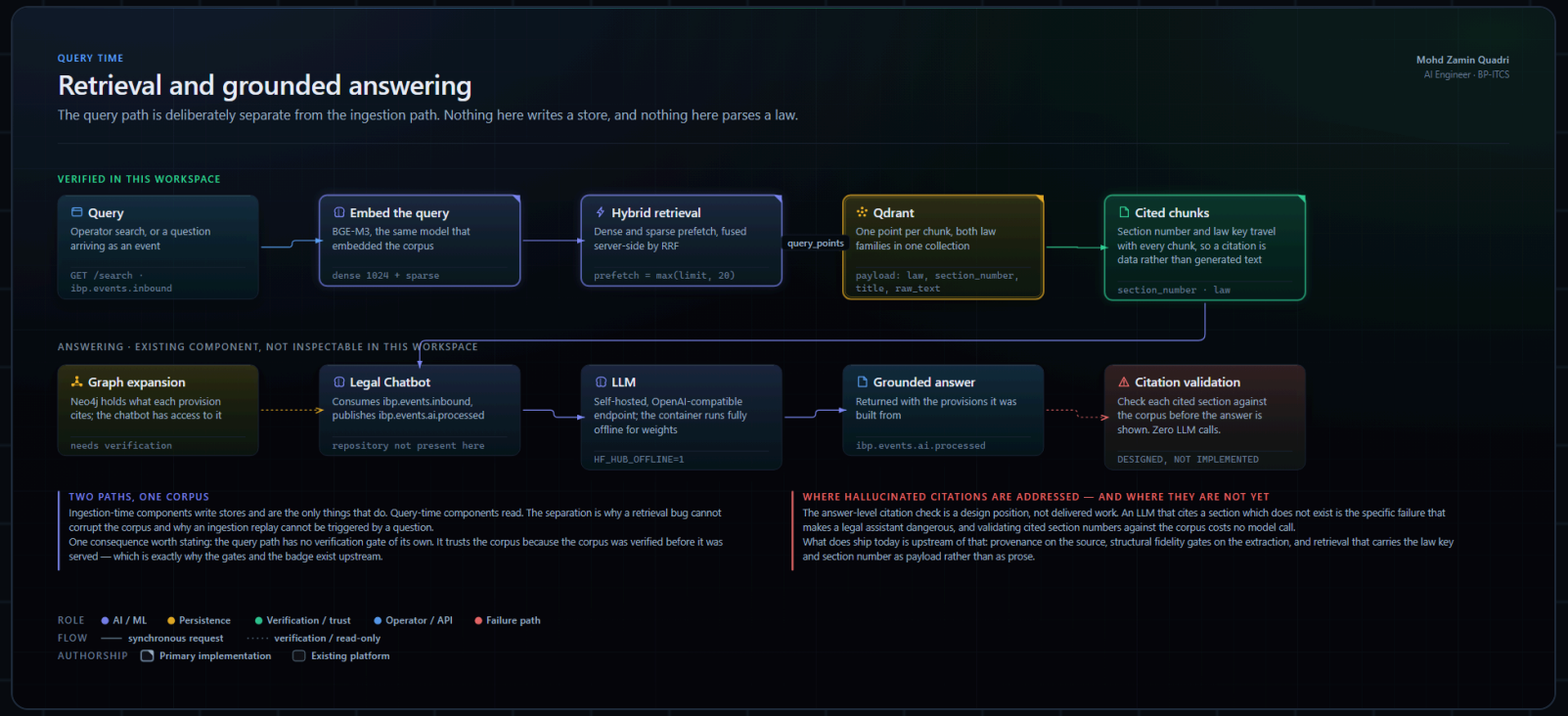
Verification and reconciliation



- 14 gates; 7 required for the badge, all 14 recorded and shown
- The prover proves itself first, or issues no verdict for any law
- A count matching is not proof — and the diagram says so

How is a question answered?

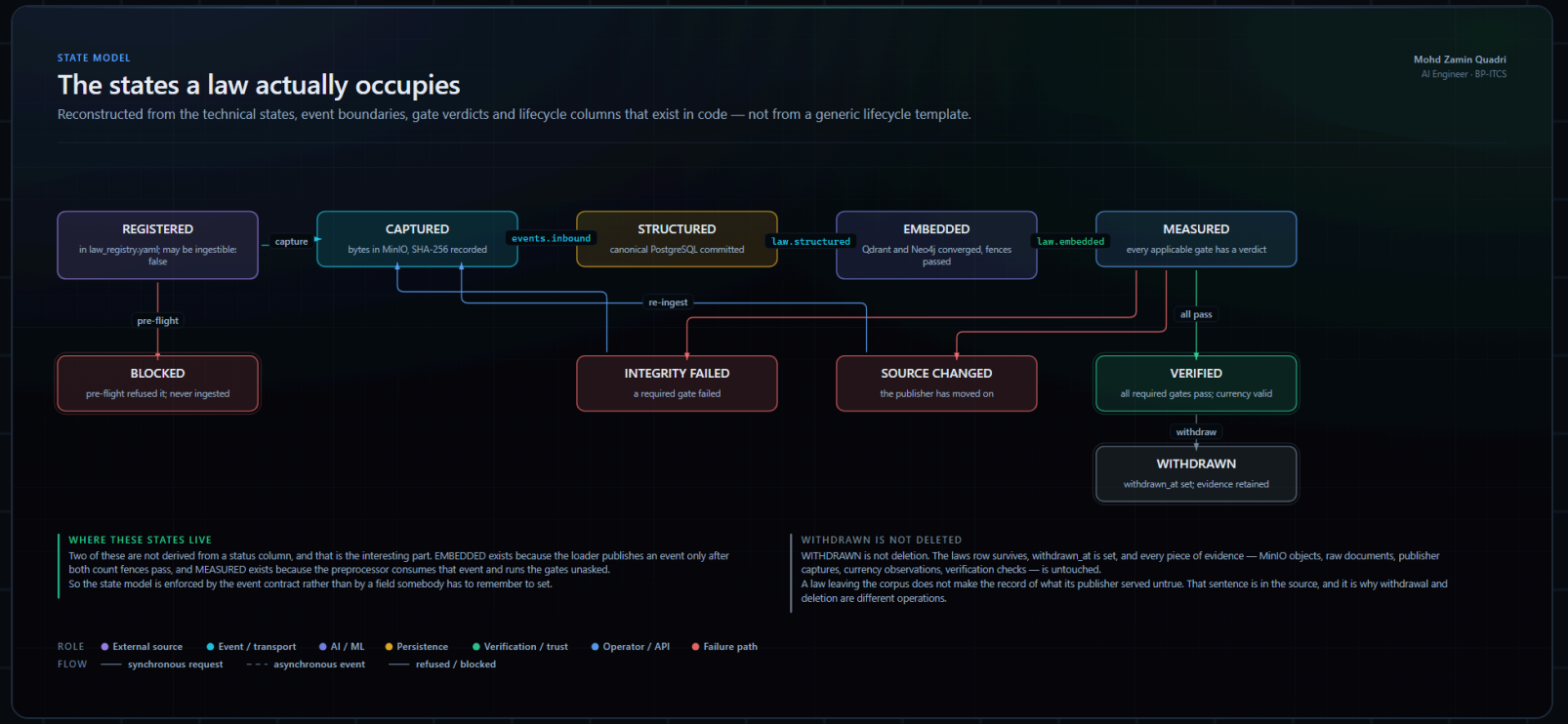
The query path



- BGE-M3 dense + sparse in one pass, fused server-side by RRF
- Citations travel as payload, not as generated prose
- The loader refuses a whole law rather than store a truncated embedding

What happens when it breaks?

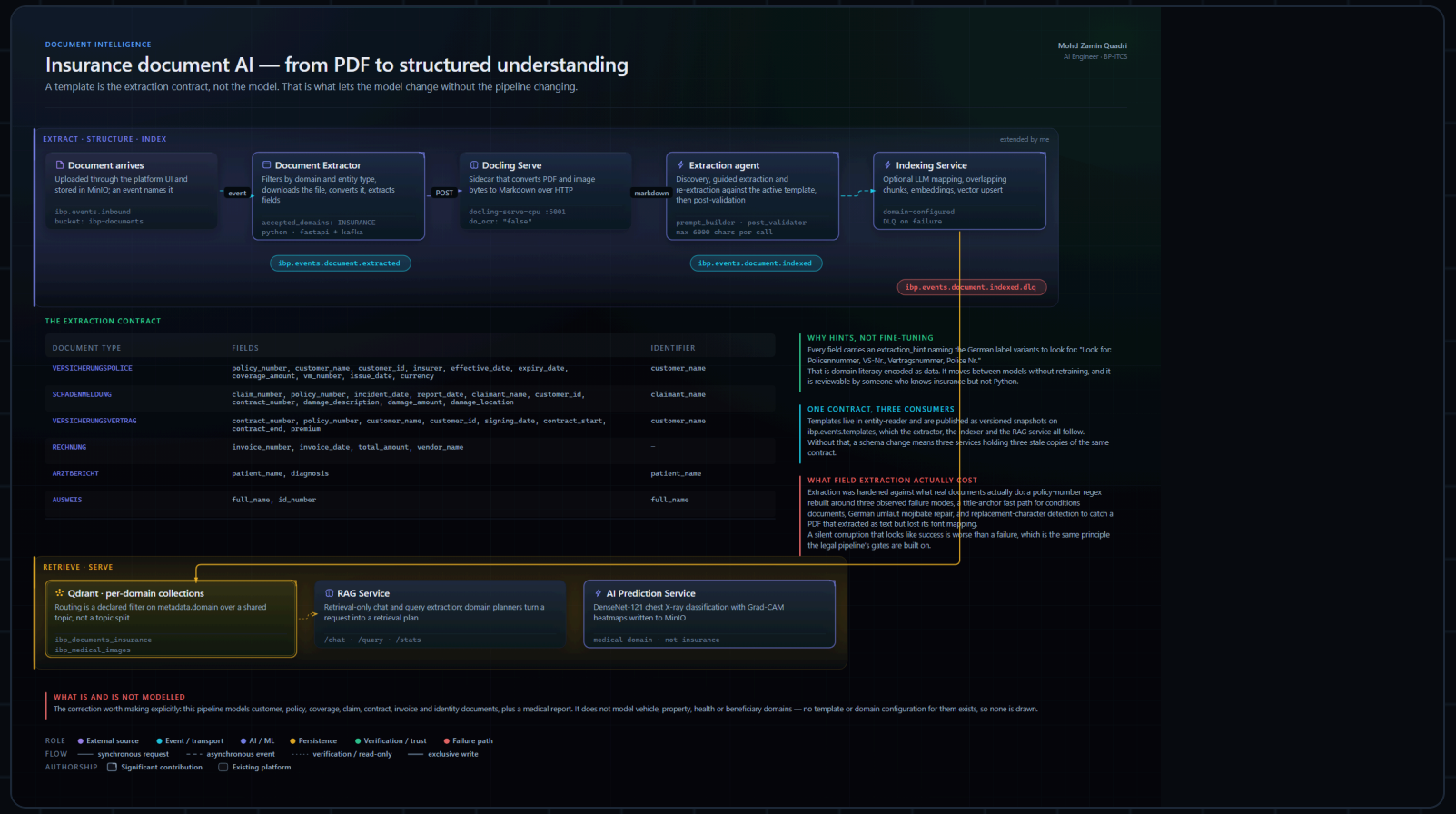
Failure, recovery and operational control



- Fifteen documented failures, each with its recovery — or the statement that none exists
- Redelivery is safe because identity is derived, never generated
- No dead-letter topic on the legal pipeline: stated as a gap, not hidden

What else runs on the platform?

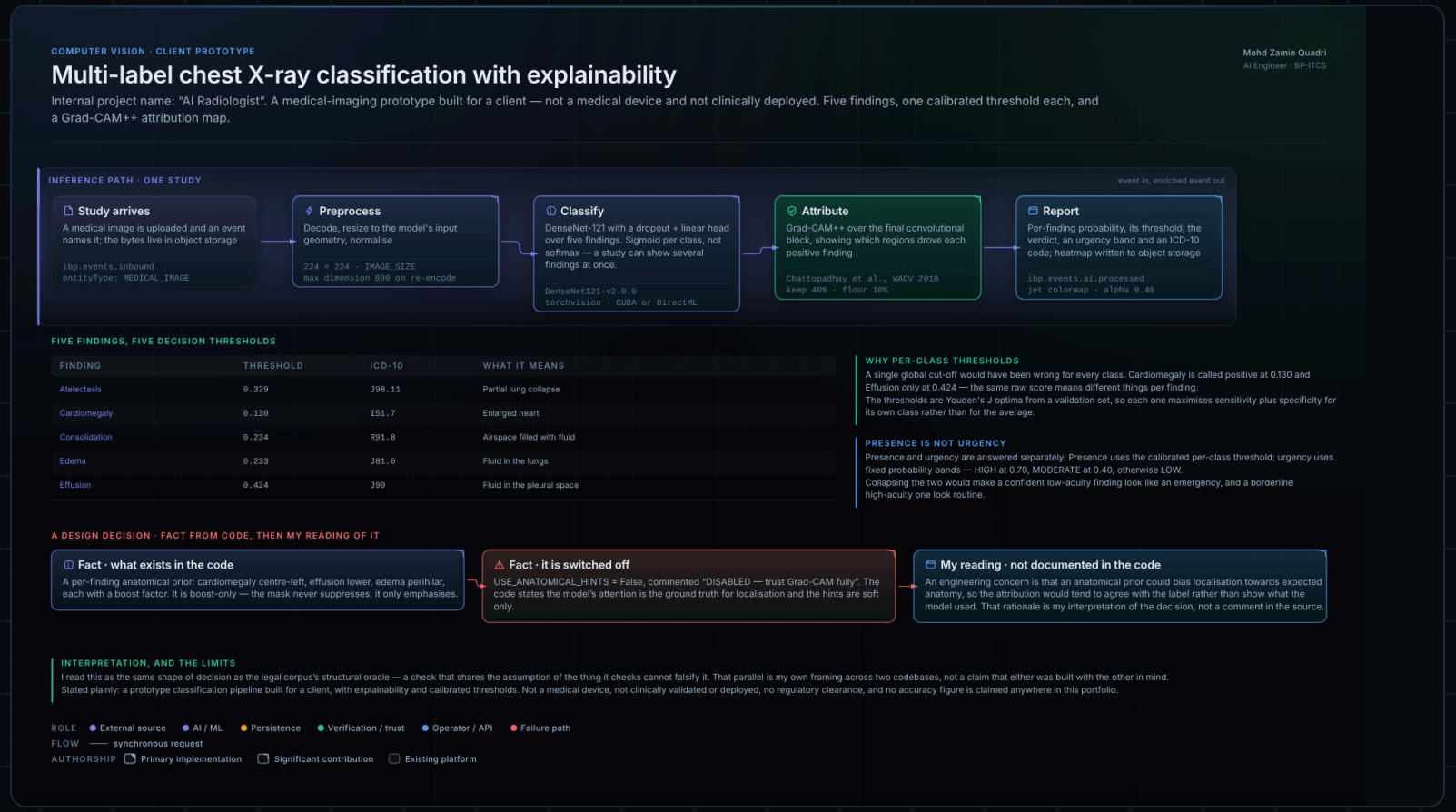
Document intelligence



- Docling conversion, then template-driven LLM field extraction with post-validation
- A template is the extraction contract, not the model — six document types
- Hardened against real documents: rebuilt regexes, umlaut repair, replacement-character detection

Is there computer vision too?

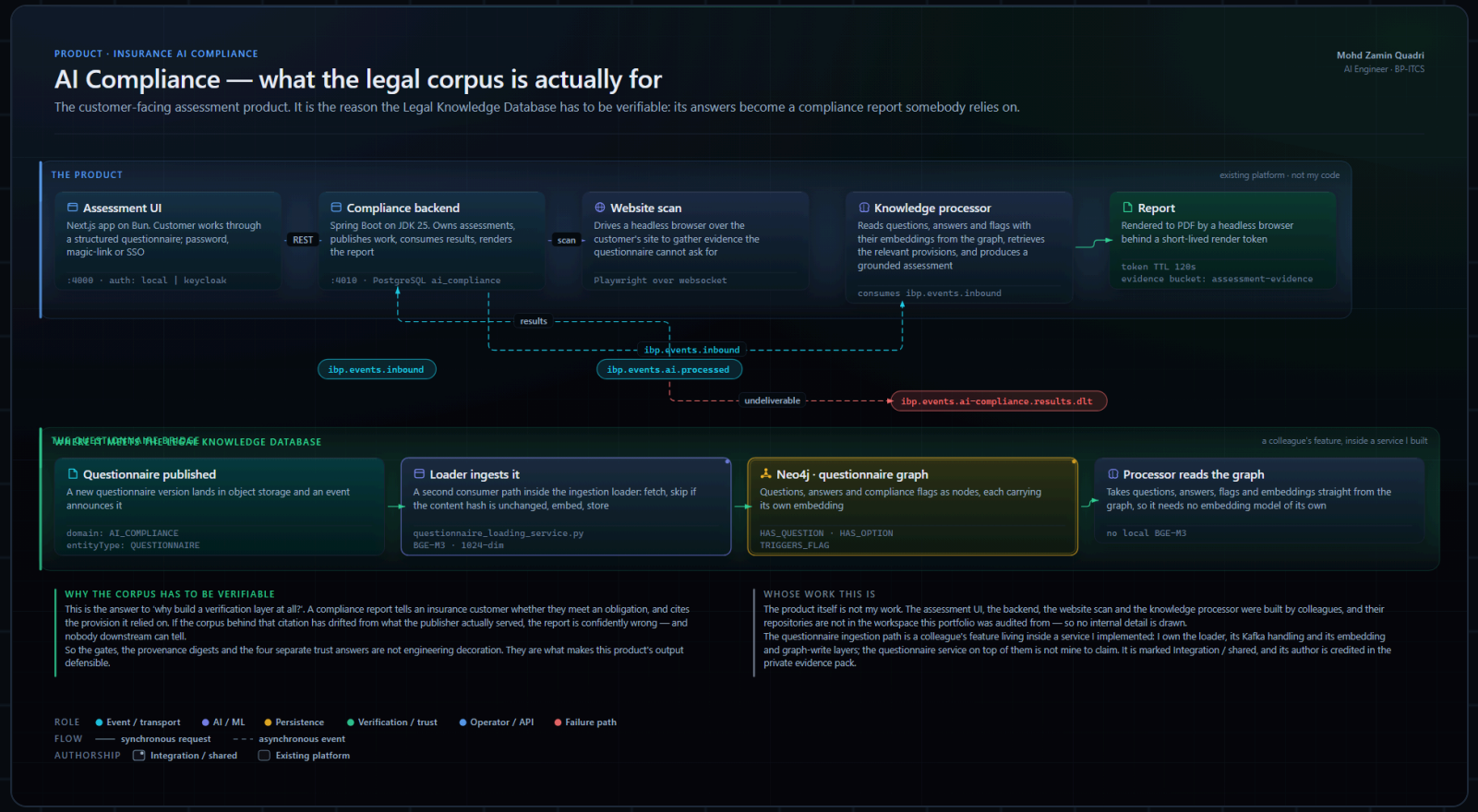
Chest X-ray classification — a client prototype



- Internal project name "AI Radiologist" — not a medical device, not clinically deployed
- DenseNet-121 on NIH, CheXpert and pneumonia data — five findings handed over, work extended to 14
- One calibrated threshold per finding (Youden's J), plus ICD-10 and an urgency band
- Grad-CAM++ attribution, with anatomical priors built and then deliberately switched off

What is all of it actually for?

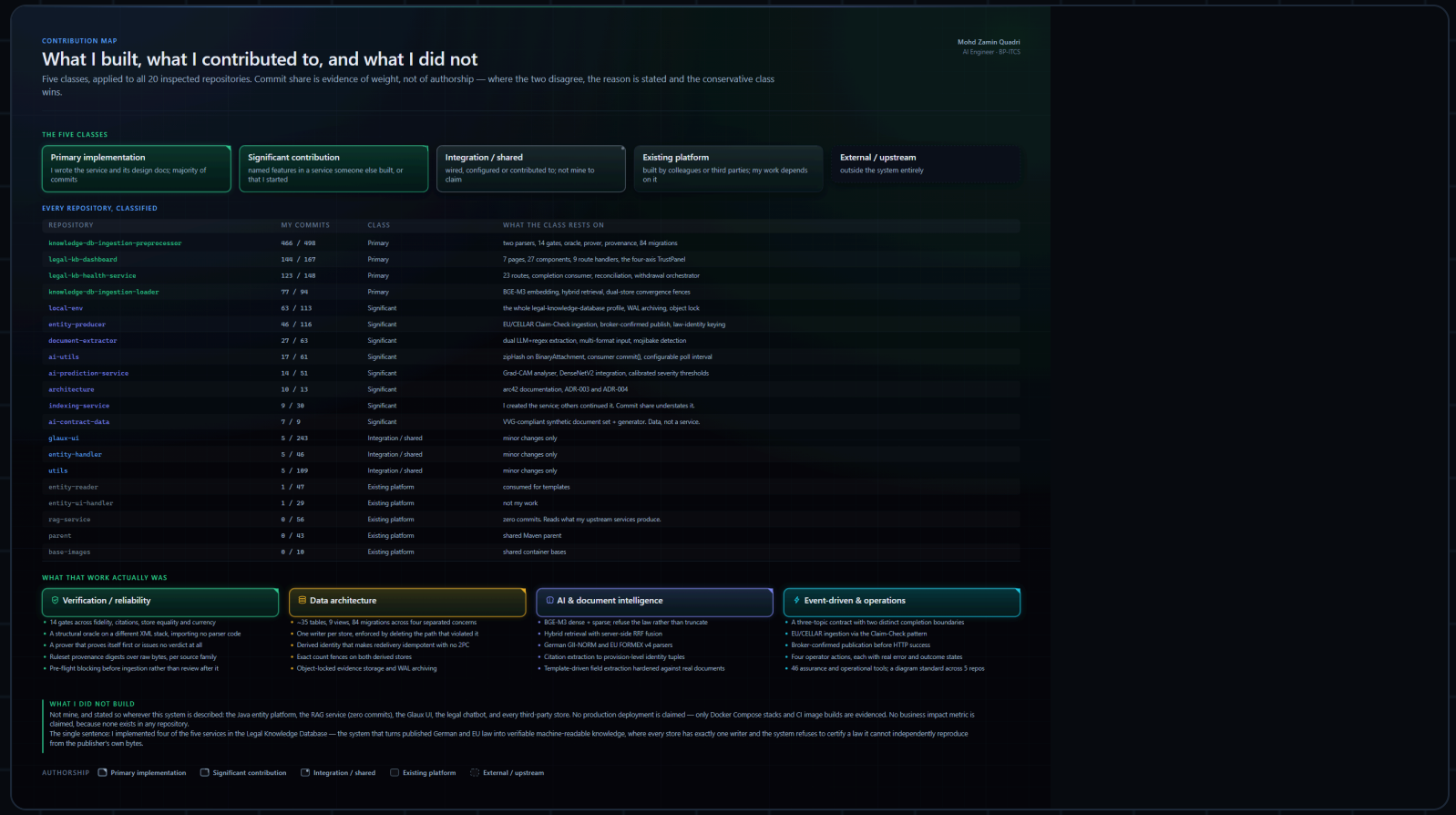
AI Compliance — the product the corpus serves



- A questionnaire plus a headless-browser site scan become a grounded, cited compliance report
- A report that cites a drifted provision is confidently wrong — which is why the corpus is verified
- The product is a colleague's work; I built the corpus and the service the questionnaire path runs in

What did you personally build?

What I built, and what I did not



CONTRIBUTION MAP

What I built, what I contributed to, and what I did not

Mohd Zamin Quadri
AI Engineer · BP-ITCS

Five classes, applied to all 20 inspected repositories. Commit share is evidence of weight, not of authorship — where the two disagree, the reason is stated and the conservative class wins.

THE FIVE CLASSES

Primary implementation	Significant contribution	Integration / shared	Existing platform	External / upstream
I wrote the service and its design docs; majority of commits	named features in a service someone else built, or that I started	wired, configured or contributed to; not mine to claim	built by colleagues or third parties; my work depends on it	outside the system entirely

EVERY REPOSITORY, CLASSIFIED

REPOSITORY	MY COMMITS	CLASS	WHAT THE CLASS RESTS ON
knowledge-db-ingestion-preprocessor	466 / 498	Primary	two parsers, 14 gates, oracle, prover, provenance, 84 migrations
legal-kb-dashboard	106 / 167	Primary	7 pages, 27 components, 9 route handlers, the four-axis TrustPanel
legal-kb-health-service	123 / 148	Primary	23 routes, completion consumer, reconciliation, withdrawal orchestrator
knowledge-db-ingestion-loader	77 / 94	Primary	BGE-M3 embedding, hybrid retrieval, dual-store convergence fences
local-env	63 / 113	Significant	the whole legal-knowledge-database profile, WAL archiving, object lock
entity-producer	46 / 116	Significant	EU/CELLAR Claim-Check ingestion, broker-confirmed publish, law-identity keying
document-extractor	27 / 63	Significant	dual LLM+regex extraction, multi-format input, mojibake detection
ai-utils	17 / 61	Significant	zipHash on BinaryAttachment, consumer commit(), configurable poll interval
ai-prediction-service	14 / 51	Significant	Grad-CAM analyser, DenseNetV2 integration, calibrated severity thresholds
architecture	10 / 13	Significant	arc42 documentation, ADR-003 and ADR-004
indexing-service	9 / 30	Significant	I created the service; others continued it. Commit share understates it.
ai-contract-data	7 / 9	Significant	VVG-compliant synthetic document set + generator. Data, not a service.
glaux-ui	5 / 243	Integration / shared	minor changes only
entity-handler	5 / 46	Integration / shared	minor changes only
utils	5 / 109	Integration / shared	minor changes only
entity-reader	1 / 47	Existing platform	consumed for templates
entity-ui-handler	1 / 29	Existing platform	not my work
rag-service	0 / 56	Existing platform	zero commits. Reads what my upstream services produce.
parent	0 / 43	Existing platform	shared Maven parent
base-images	0 / 10	Existing platform	shared container bases

WHAT THAT WORK ACTUALLY WAS

Verification / reliability
- 14 gates across fidelity, citations, store equality and currency
- A structural oracle on a different XML stack, importing no parser code
- A prover that proves itself first or issues no verdict at all
- Ruleset provenance digests over raw bytes, per source family
- Pre-flight blocking before ingestion rather than review after it

Data architecture
- ~35 tables, 9 views, 84 migrations across four separated concerns
- One writer per store, enforced by deleting the path that violated it
- Derived identity that makes redelivery idempotent with no 2PC
- Exact count fences on both derived stores
- Object-locked evidence storage and WAL archiving

AI & document intelligence
- BGE-M3 dense + sparse; refuse the law rather than truncate
- Hybrid retrieval with server-side RRF fusion
- German GI-NORM and EU FORMEX v4 parsers
- Citation extraction to provision-level identity tuples
- Template-driven field extraction hardened against real documents

Event-driven & operations
- A three-topic contract with two distinct completion boundaries
- EU/CELLAR ingestion via the Claim-Check pattern
- Broker-confirmed publication before HTTP success
- Four operator actions, each with real error and outcome states
- 46 assurance and operational tools; a diagram standard across 5 repos

WHAT I DID NOT BUILD

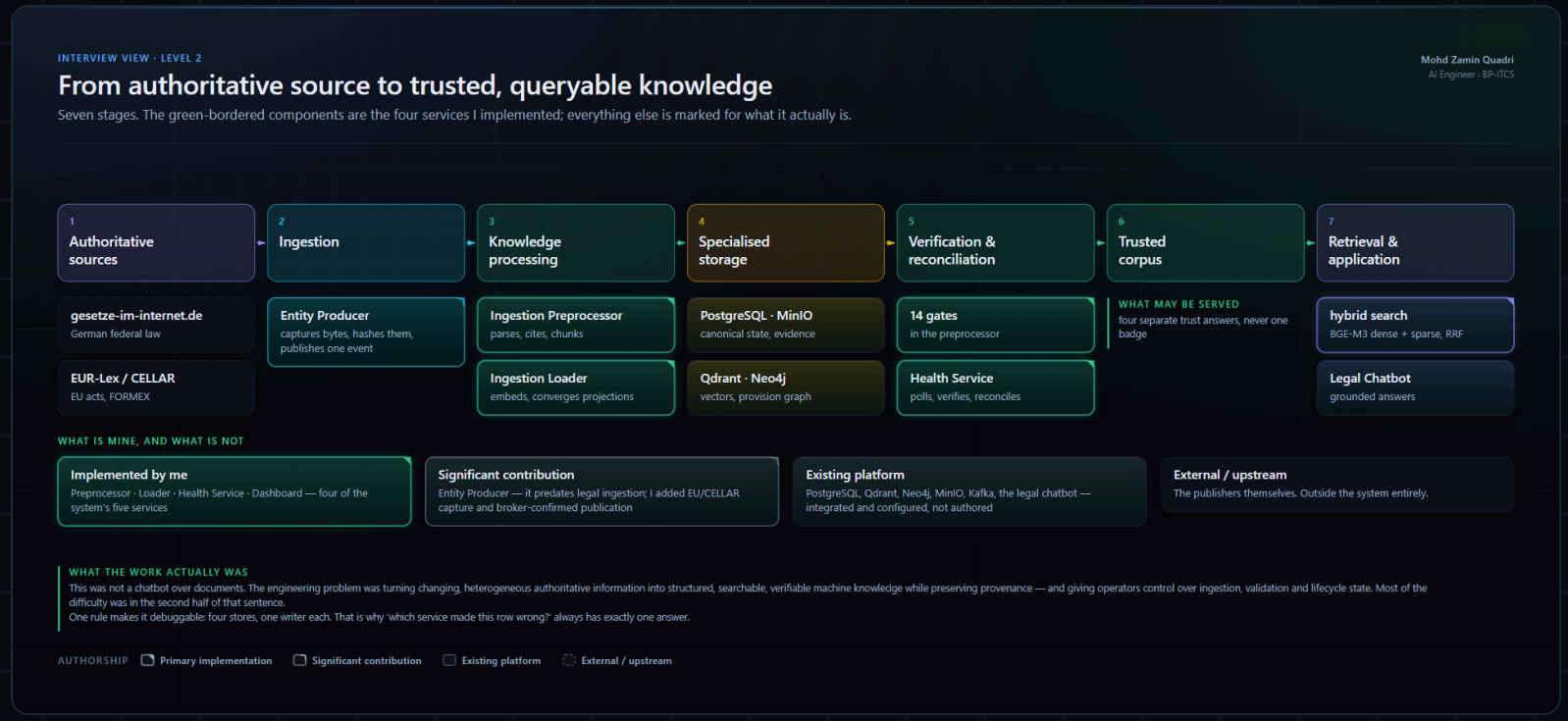
Not mine, and stated so wherever this system is described: the Java entity platform, the RAG service (zero commits), the Glaux UI, the legal chatbot, and every third-party store. No production deployment is claimed — only Docker Compose stacks and CI image builds are evidenced. No business impact metric is claimed, because none exists in any repository.
The single sentence: I implemented four of the five services in the Legal Knowledge Database — the system that turns published German and EU law into verifiable machine-readable knowledge, where every store has exactly one writer and the system refuses to certify a law it cannot independently reproduce from the publisher's own bytes.

AUTHORSHIP Primary implementation · Significant contribution · Integration / shared · Existing platform · External / upstream

- Primary: preprocessor, loader, health service, dashboard — four of the system's five services
- Significant: entity producer, document extractor, indexing service (which I created), ai-utils
- Not mine: the Java entity platform, the RAG service, the chatbot, every third-party store

What would you do differently?

Engineering lessons and design principles



- Keep: one writer per store; derive identity; refuse rather than approximate
- Change: add a dead-letter path; generate the gate list once; content-address the captures
- The lesson: a measurement nobody can falsify is not a measurement